

Evolutionary Strategies

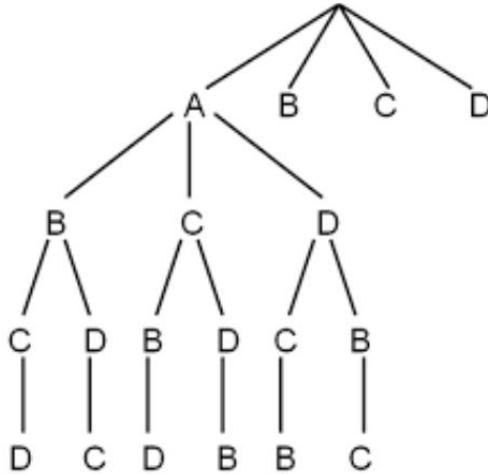
Last session:

- Search as a strategy for efficient credit assignment
- Two ways of thinking about search:
 - **Von Neumann search**: playing a well-defined game
 - **Semantic information**: structuring an ill-defined problem

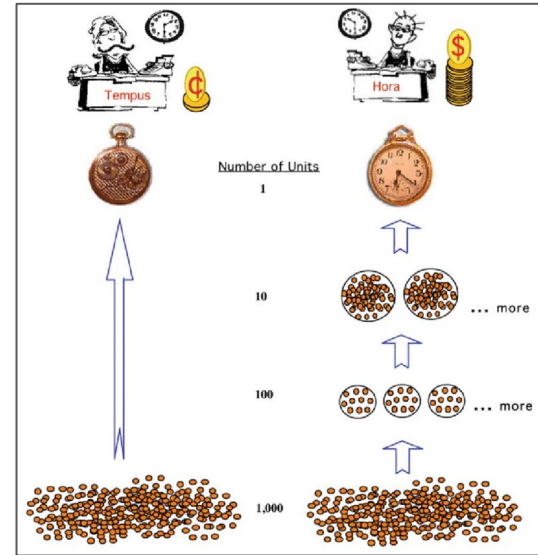
This session:

- What happens when problems are too complex for basic search?
- Evolutionary strategies = ways of making search more **effective** (or even avoiding the need to search) based on **adaptive processes**

Our key move: shifting from focus on **sequential** interdependence to **spatial** interdependence



Basic search: mostly about sequences of actions



Evolutionary strategies: mostly about how actions are spatially (functionally) related

We'll think of **plans** as more like schemas or representations of problems

- Blueprints, schematics, etc.: describe how building blocks fit together
- Plans “evolve”, similar to Darwin’s ideas about evolution.

Two paradigms:

- **Genetic approach**: building blocks of AI systems and orgs. as like genes or chromosomes (closer to reinf. learning & Shannon info.)
- **Biological systems information**: building blocks of AI systems and orgs. as like complex cells (closer to learning-in-a-society, and Simon info.)

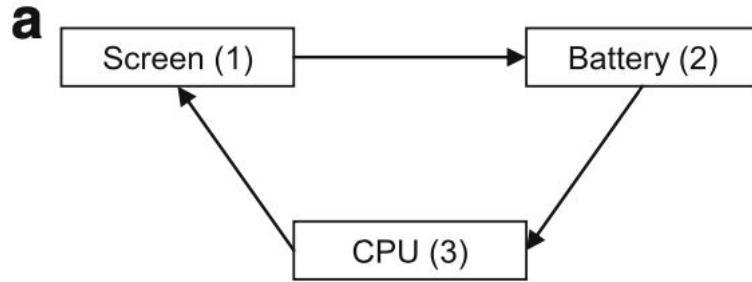
Evol. strategies = view of credit assignment as an **adaptive process**

- Adaptation = modifying one's beliefs/genes/actions etc. based on evolutionary operators (VSR):
 - **Variation** (generating new beliefs/genes/actions)
 - **Selection** (assigning credit to high-performing ones)
 - **Retention** (using these ones in the next “generation”)

In evolutionary strategies, intelligence = “**fitness**”

- Fitness:
 - Biology: ability to survive and reproduce
 - AI systems and orgs.: relative quality of a solution to a problem
- Fitness is always *relative to some problem-solving environment*
 - Polar bear has high fitness in arctic, low fitness in tropics
 - High fitness **does not = intelligence**, but is often a useful proxy.

Example | Designing a laptop of three components



Directed graph

b

$$\mathbf{I} = \begin{matrix} & \begin{matrix} 1 & 2 & 3 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} & \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix} \end{matrix}$$

Interaction matrix

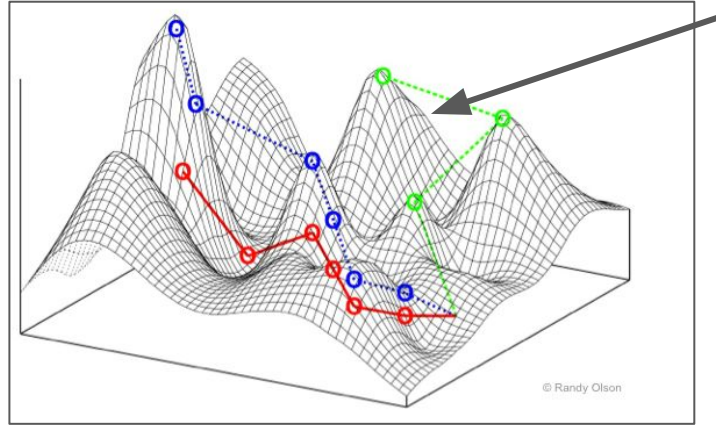
Contribution function = how each action contributes to performance (i.e., elevation)

Contribution of screen	Contribution of battery	Contribution of CPU
<u>screen</u> <u>CPU</u>	<u>battery</u> <u>screen</u>	<u>CPU</u> <u>battery</u>
$c_1(\text{ 0 } , \text{ 0 }) = 0.9$	$c_2(\text{ 0 } , \text{ 0 }) = 0.8$	$c_3(\text{ 0 } , \text{ 0 }) = 0.1$
$c_1(\text{ 0 } , \text{ 1 }) = 0.5$	$c_2(\text{ 0 } , \text{ 1 }) = 0.0$	$c_3(\text{ 0 } , \text{ 1 }) = 0.5$
$c_1(\text{ 1 } , \text{ 0 }) = 0.3$	$c_2(\text{ 1 } , \text{ 0 }) = 0.0$	$c_3(\text{ 1 } , \text{ 0 }) = 0.2$
$c_1(\text{ 1 } , \text{ 1 }) = 0.7$	$c_2(\text{ 1 } , \text{ 1 }) = 0.3$	$c_3(\text{ 1 } , \text{ 1 }) = 0.9$

Thus, the fitness of a portable computer with small screen ($s_1 = 0$), weak battery ($s_2 = 0$), and fast CPU ($s_3 = 1$) would be:

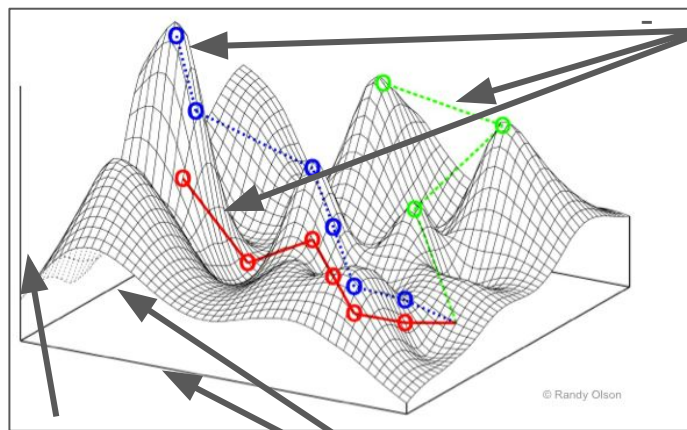
$$f(0, 0, 1) = \frac{1}{3}(c_1(0, 1) + c_2(0, 0) + c_3(1, 0)) = \frac{1}{3}(0.5 + 0.8 + 0.2) = 0.5.$$

Performance landscapes = visualizing the contribution function as a mapping of fitness values to points on a landscape



Each point on the landscape = a fitness value = the value of a particular solution to a problem.

Performance landscapes visualize adaptively searching for solutions to problems as a process of “navigating” the surface of the landscape



Paths =
learning
processes
resulting from
different actions

- **Y-axis = performance**
("elevation" of the landscape)

X-axes = actions (“navigating” the landscape)

Elements of landscapes

- (1) **Performance goal** | Fitness, or relative performance of solutions
- (2) **Set of possible actions** | Choices, observations, etc.
- (3) **Set of possible states** | Outcomes of actions
- (4) **Selection procedures for actions** | How to use feedback to select actions
- (5) **Procedures for selecting (4)** | E.g., setting the learning rate.

Landscape topologies

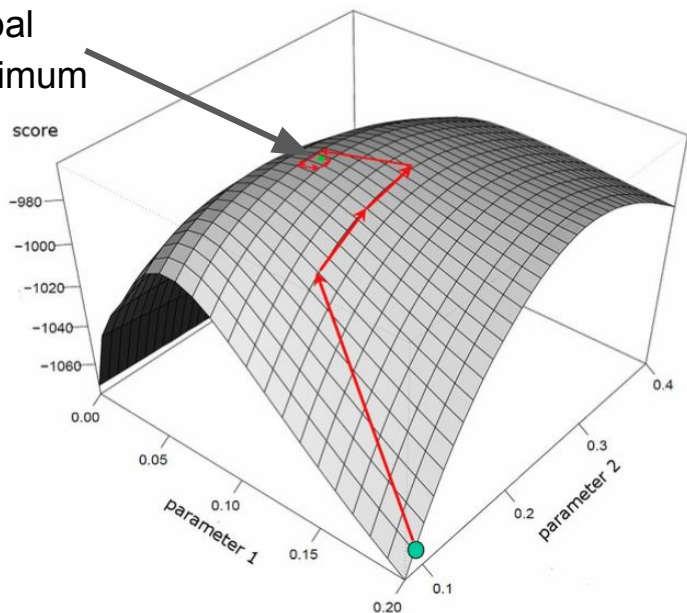
“**Hills**” = “gradients” = areas where trial-and-error learning is possible

“**Peaks**” = optimal points in a landscape

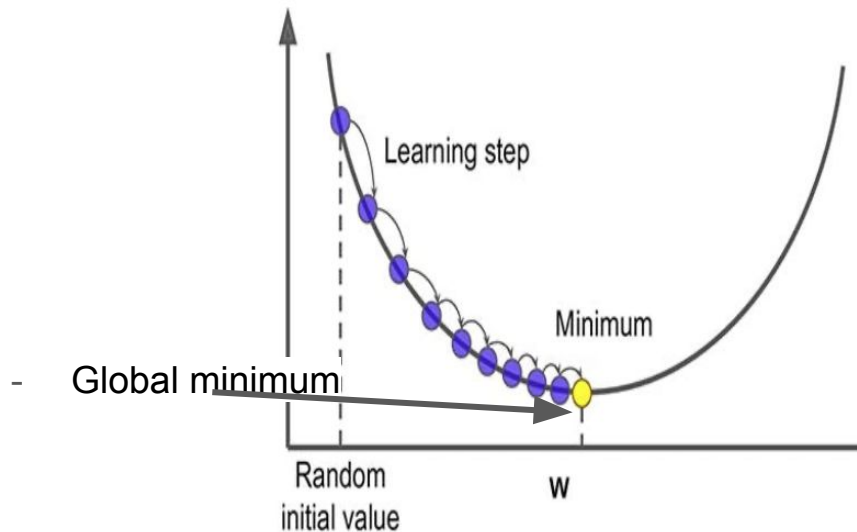
“**Mesas**” = satisfactory thresholds in the landscape

Single-peaked landscapes (hill-climbing/gradient descent)

- Global maximum

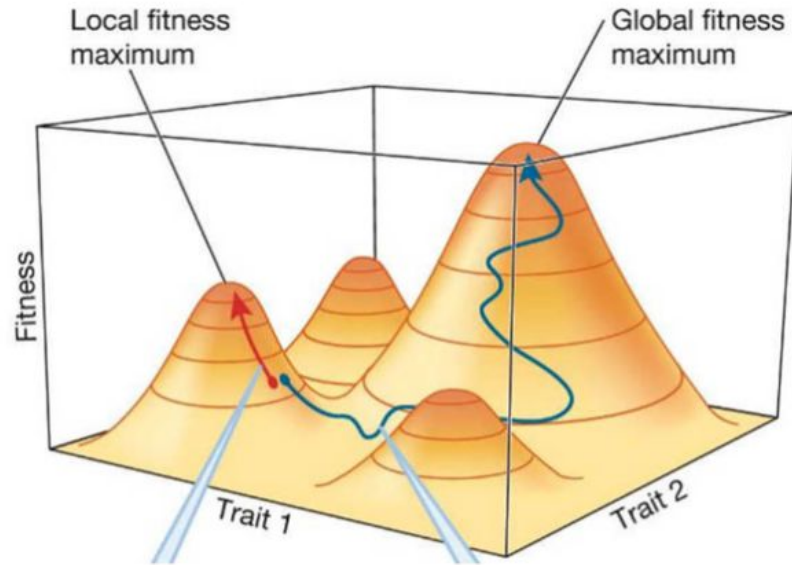


Hill-climbing = maximizing reward

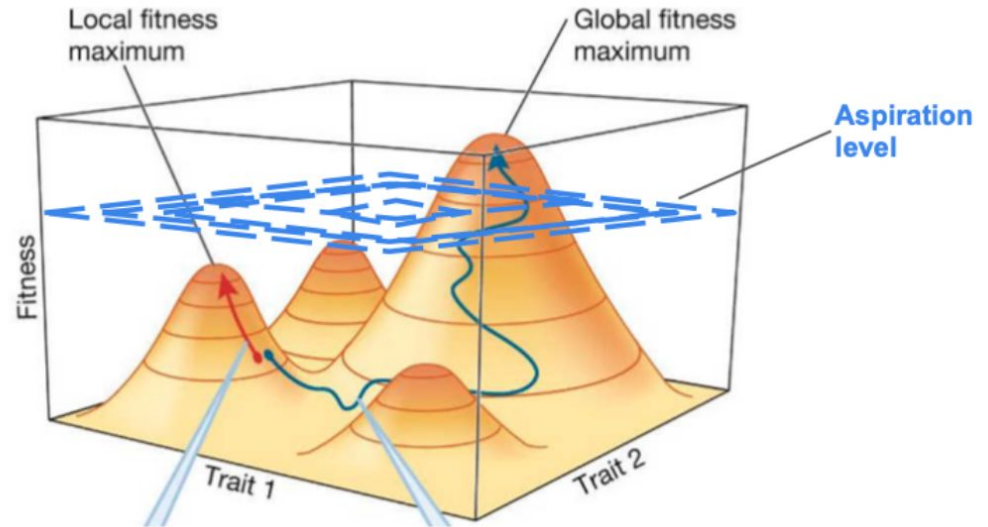


Gradient descent = minimizing error

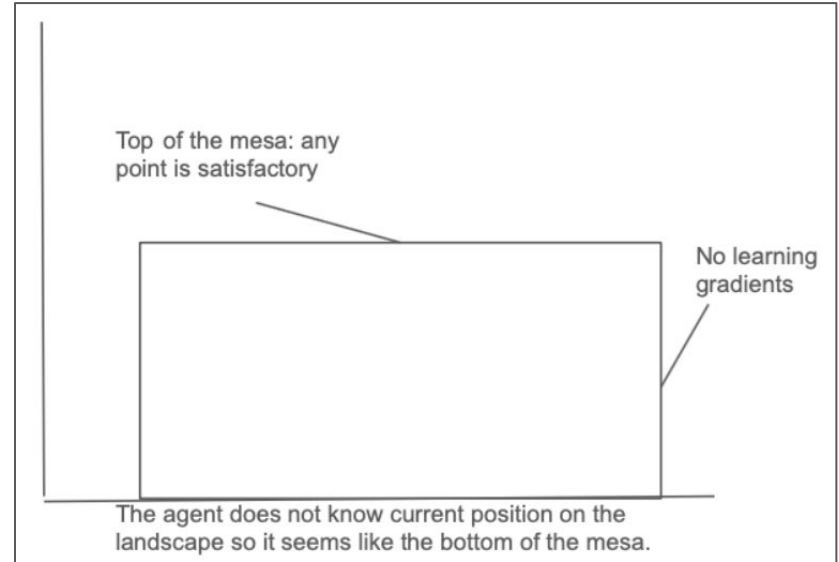
Rugged landscapes (balancing hill-climbing and long-jumping)



Satisficing in a rugged landscape (lowering expectations)



Satisficing in a mesa landscape (taking giant strides)



Landscape topologies lead to different learning challenges

Landscape topology	Landscape features	Learning challenges	Learning strategies
Optimizing in single-peaked landscapes	One global peak, otherwise “smooth”	If interdependencies are large, vast info. processing required	Hill-climbing / gradient descent
Optimizing in rugged landscapes	Multiple local peaks	Myopia, path dependence, superstitious learning	Balance exploration (long-jumping) and Exploitation (hill-climbing)
Satisficing in rugged landscapes	Aspiration below global peak but above some local peaks	Setting a realistic aspiration level	Use heuristics to set an aspiration level
Satisficing in mesa landscapes	A flat bottom and a flat top, with no hills or gradients in between	Problems arise only as symptoms, where there are a lack of gradients for trial-and-error learning (sparse learning)	Taking “ giant strides ” by logically and coherently piecing together the problem.

Evolutionary strategies

(1) Genetic approach (Shannon info.)

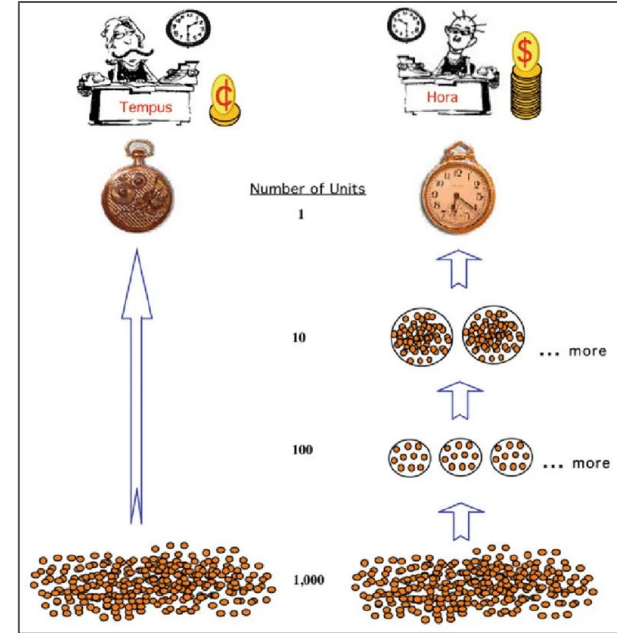
- 1a. Artificial selection environment (Mendelian experimentation)
- 1b. Genetic algorithms (John Holland, U. Michigan)

(2) Biological systems approach (Simon info.)

- 2. Propagator model (Gerald Sussman, MIT)

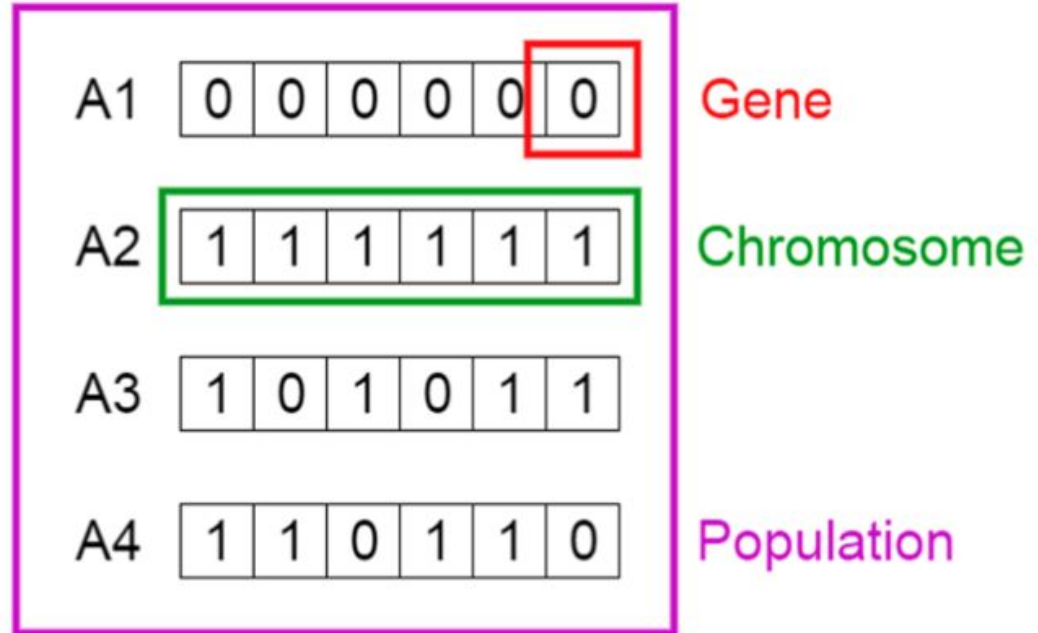
Evolutionary strategies = “smoothing” the landscape

- How? Decomposing problems = identifying “**constraints**” = areas where zero information is needed to adapt/learn to assign credit.
- Simon’s second watchmaker:



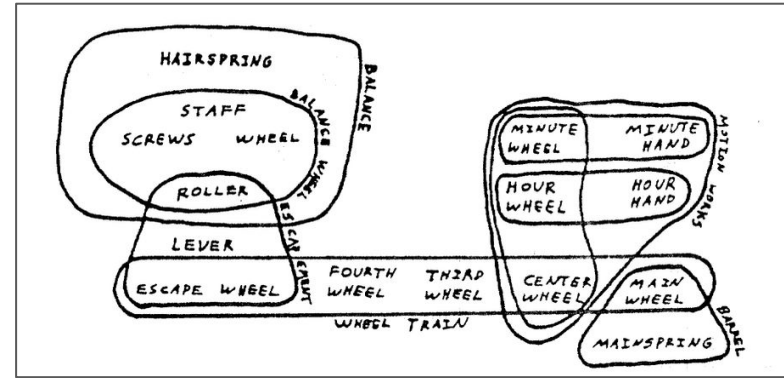
Genetic approach (consistent with rugged landscapes)

- Actions in an org./AI system as like **genes** in a **chromosome**.
- Populations of chromosomes (analogous to product designs, computer programs, etc.) compete across generations.

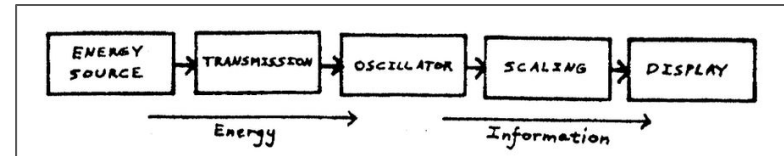


Biological systems approach (consistent with mesa landscapes)

- Actions in an org./AI system as like **functions** in a **biological organism**.
- Functions can be fulfilled by diverse genes/chromosomes, and the same gene/chromosome can be used for diverse functions.
- Evolutionary strategy = start with a **functional structure**, then adapt various parts and components to it. Like how cells grow in organisms.



Anatomy of the watch



Functional structure of the watch

Biological systems approach (consistent with mesa landscapes)

The “evolved” watch decomposes the parts and components according to **constraints** on how it functions.

